

SLOT - C

2

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1402	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	Al najeeb	Reg. No.:	192424325
Section / Batch:	AI BDS	Date:	16/7/2026

Code of Conduct

I Al najeeb 192424325 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Al najeeb

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	4 / 4	3 / 4	91.25
2	4 / 4	4 / 4	3 / 4	4 / 4	4 / 4	93.75
3	3 / 4	4 / 4	3 / 4	4 / 4	4 / 4	88.75
4	4 / 4	4 / 4	3 / 4	3 / 4	4 / 4	88.75
5	3 / 4	3 / 4	3 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	90.5
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Al najeeb

Assessment

tool

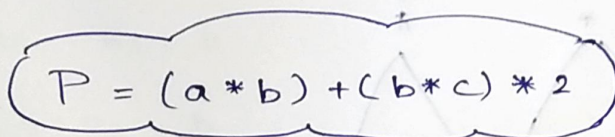
CSA 1402

— Alnajeel . A
192424306

Trace the processing of the input expression

$P = (a * b) + (b * c) * 2$ through all phases of compiler

- List all tokens
- Construct the Syntax tree
- Generate the intermediate code
- Apply Suitable Optimization
- Final target code


$$P = (a * b) + (b * c) * 2$$

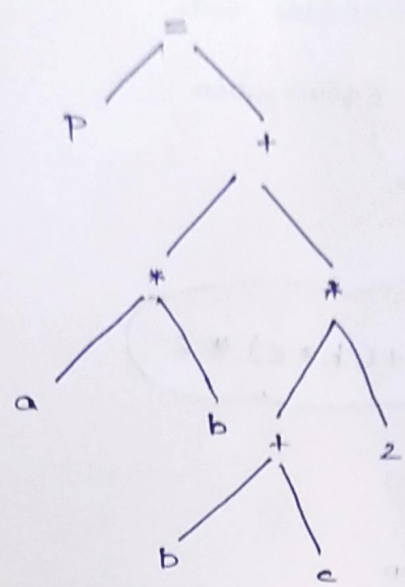
a) Lexical Analysis :

Lexeme	Token
P	id1
=	Assign operator
(	Parenthesis
a	id2
*	multiplication Op
b	id3
)	Parenthesis
+	Add Op
(	Left parenthesis
b	id4
*	mul Op

- (id's
-) Right parenthesis
- * mul Ops
- 2 Constant

b)

Syntax tree



c

Intermediate Code:

- $t_1 = a * b$
- $t_2 = b * c$
- $t_3 = t_2 * 2$
- $t_4 = t_1 + t_3$
- $P = t_4$

Optimized Intermediate Code:

2103-C

$$t_1 = a * b$$

$$t_2 = b * c$$

$$P = t_1 + (t_2 * 2)$$

c)

MOV R1, a

MUL R1, b

MOV R2, b

MUL R2, c

MUL R2, #2

ADD R1, R2

MOV P, R1

Conclusion :

* The final machine, assembly code can be executed directly by the processor. Uses CPU register efficiently. produces faster execution

2. For each of the following RE construct 5 valid strings of each RE

a)

Regular Expression: $(\epsilon + aa^*)^*$

1. ϵ

2. a

3. aa

4. aaa

5. aa aa

b)

Regular Expression: $(a + b^*)abb$

1. aa bb

2. aaa bb

3. ab abb

4. aaa aba bb

5. aab bbb abb

$$(a^* b^*)^* aba (a^* + b^*)^*$$

1. aba
2. aaba
3. abab
4. aaaa babb
5. baba baa.

d)

$$(a + ab)^*$$

1. ϵ
2. a
3. ab
4. abab
5. aa

e)

$$(aba) + (a^* b^*)^*$$

1. aba
2. ab aaba
3. abaa
4. abab
5. abaaabb.

3.

- (i) Find the number of tokens in the following C statement is :

```
main ()  
{  
    int a = 10;  
    char b = "abc";  
    int c = 30;  
    char d = "xyz";  
    int m = 40.5;  
}
```

Tokens	Count
Main	1
(	1
)	1
{	1
int	2
a	1
=	5
10	1
;	5
Char	2

"abc"

c

30

d

"xyb"

m

40.5

y

Token total Count : 30

Lexical Error:

Void ()

{

int x = 16 , y = 20 ;

char * a ;

a = 0x ;

x = 1xab ;

}

3
(ii)

Errors:

1. $a = 0x;$

* a is a character pointer while $0x$ is an integer address

* In Valid assignment

2. $1xab$

* Invalid Identifier

* Identifier cannot start with a digit.

* Lexer reports an invalid tokens.

4. Find a regular expression corresponding to each of the following subset of $(0,1)^*$:

(i) The language of all string that have an even number of 0's.

(ii) The language of all string that contains at least one 1.

(iii) The language of all string in which both the number of 0's & 1's

(iv) The Language of all string that start with 1 and end 0.

The language of all string that do not contain consecutive 1's

(i) Even number of 0's

Regular expression: $1^* (01^* 01^*)^*$

(ii) Contains at least one 1

Regular Expression: $0^* 1 0^*$

Regular Expression: $(0+1)^* 1 (0+1)^*$

(iii) Both number of 0's and number of 1's are odd.

Regular Expression:

$0(00)^* 1(11)^* (00+11+01+10)^*$

(iv)

Starts with 1 & ends with 0

Regular Expression: $1(0+1)^* 0$

(v) Does not contain consecutive 1's

$(0+10)^* (\epsilon+1)$

The lexical analyzer needs to tokenize the following input string:

int Var123 = 12345, float Var067 = 123.45; if (Var123 == Var067)

{ return 3;

(i) How many tokens?

(ii) Provide a list of tokens categorized by their types.

(iii) Drawing a FSA is used for lexical analysis.

Estimating number.

Token	Type
int	Keyword (1)
Var123	Id
=	operator
12345	Integer
,	Separator
float	Keyword
Var067	Identifier
.	Operator

5.

The lexical analyzer needs to tokenize the following input string :

```
int Var123 = 12345, float Var456 = 123.45; if (Var123 == Var456)
{ return 3;
```

- How many tokens ?
- Provide a list of tokens categorized by their types.
- Assuming a FSA is used for lexical analysis estimating number.

a)

Token	Type
int	Keyword (1)
Var123	Id
=	operator
12345	Integer
;	Separator
float	Keyword
Var456	Identifier
=	Operator

12.5.46

}	Separator
if	Keyword
(	Separator
Var123	id
==	Operator
Var456	id
)	Separator
{	Separator
return	Keyword
,	Separator
;	Separator

Total tokens 20

(b)

Categorization:

Keyword (4)

- * int
- * float
- * if
- * return

Operator (5)

- * =
- * =
- * ==

Separator (7)

- *)
- *)
- * (
- *)
- * {
- * }
- * ;

Identifier (11)

- * Var123
- * Var123
- * Var123
- * Var123

Integer Literal (1)

12345

Float Literal:

123.45

B
C

Estimate the number of FSM:

Token

Approx. Transition:

int	3
Var123	6
=	1
12345	5
;	1
float	5
Var123	6
=	1

123.45

,

6

if

1

(

2

Var123

1

==

6

2

Var1456

6

)

1

{

1

return

6

,

1

}

1

Total FSH Transition = 70.

Conclusion :

* Every character read causes one state transition in FSH, keyword (int, float, if, return) are recognised letter by letter.

RUBRIC FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear interpretation or	No interpretation